Lesson plan · for the teacher

Automate the weekly report

Data engineering · Level 4 · Lesson 1

Time
40 min

Materials
Three made-up class registers on paper; Board and chalk; Optional: a computer or phone with Python

Learning goals

- Describe a pipeline: extract, transform, load.
- Make every run safe to repeat.
- Tell passing faults from bad data.

Activity: The human pipeline

1. Warm-up (5 min): Ask how the school's weekly attendance report is put together today, who does it and how long it takes.
2. Teach (8 min): Draw the three steps and a schedule on the board. Explain passing faults and bad data, and why the load replaces the day's rows.
3. Run (12 min): One group reads three made-up registers aloud, one counts the P's per class, and one writes the counts on the board. Put a "?" in one register.
4. Fail (10 min): Stop the writers after two classes, then run again. First add the rows, then try wiping the day's rows first. Which gives the right total?
5. Wrap-up (5 min): Agree what the run log should say, including the "?". Then give the exit ticket.

Use made-up registers. Make the class sizes add up to a number pupils can check, such as 90, so a double count is easy to spot.

Discussion

- Why did adding the rows again give 152 pupils?
- Which failures will a retry never fix?
- What should the run log tell the head?

Exit ticket

- Write one passing fault and one bad-data fault, and what the pipeline should do with each.

Try it at school tomorrow
Run a "Human pipeline": one group reads the registers aloud, one counts, and one writes the counts on the board. Then stop it halfway and run it again: did anyone count a class twice?

Practice questions · for you

Automate the weekly report

Data engineering · Level 4 · Lesson 1

1 **A register says "?" instead of P or A, and the transform step stops. Will a retry fix it?**

◯ Yes, if the network is faster ◯ Yes, retry until it works ◯ No: bad data needs a person to check it

2 **Put the steps of the evening's attendance pipeline in order.**

☐ Transform: count the pupils present in each class

☐ Extract: read the three class registers

☐ Log: write what ran, and what was set aside

☐ Load: replace today's rows in the district's table

3 **The transform step. register.csv has one row per pupil: class, pupil number and P (present) or A (away). Fix the program so it counts only the pupils present in each class, printed in class order: 5A 4, 5B 3, 6A 5.**

```
import csv
present = {}
with open("register.csv") as f:
    for row in csv.DictReader(f):
        c = row["class"]
        present[c] = present.get(c, 0) + 1
for c in sorted(present):
    print(c, present[c])
```

Write your program:

4 **Put the lines in order (and indent them): an idempotent load that replaces the day's rows in the report instead of adding to them.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn't belong.

☐ `report.append((day, cls, n))`

☐ `report = report + counts`

☐ `report = [r for r in report if r[0] != day]`

☐ `return report`

☐ `def load(report, day, counts):`

☐ `for cls, n in counts:`

5 **Retrying a failed step fixes any fault, as long as you retry enough times.**

◯ True ◯ False

✎ **Which report at your school is put together by hand every week or term? Sketch it as extract, transform and load.**

Check your answers

Automate the weekly report

Data engineering · Level 4 · Lesson 1

1 **No: bad data needs a person to check it**

The data itself is wrong, so every retry fails the same way.

2 **Top to bottom: 2, 1, 4, 3**

Extract, transform, load, in that order, and then the log, so it can record what every step did, including any rows set aside for a person to check.

3

```
import csv
present = {}
with open("register.csv") as f:
    for row in csv.DictReader(f):
        if row["status"] == "P":
            c = row["class"]
            present[c] = present.get(c, 0) + 1
for c in sorted(present):
    print(c, present[c])
```

present.get(c, 0) + 1 adds one to the class's count, starting from 0 the first time.

4 **Top to bottom: 4, X, 2, 5, 1, 3**

```
def load(report, day, counts):
    report = [r for r in report if r[0] != day]
    for cls, n in counts:
        report.append((day, cls, n))
    return report
```

First drop every row already in the report for that day, then add the day's counts, and return the new report.

5 **False**

Retries fix passing faults, such as a dropped connection.